

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dvOretsky/fastapi-tutorial/blob/main/FAPI6_Authentication/FAPI6_Authentication.md

Практическое занятие №6: Реализация аутентификации

Аутентификация — это процесс проверки личности пользователя или системы, пытающихся получить доступ к веб-приложению. Другими словами, аутентификация обеспечивает подтверждение того, что перед нами действительно тот, за кого себя выдают.

Чтобы понять этот процесс, представьте ситуацию: вам в мессенджере пишет человек, утверждая, что он ваш друг, и просит занять денег. Однако вы знаете, что настоящий друг сейчас в отпуске без интернета. Чтобы убедиться, что это действительно он, вы задаете вопрос, на который знает ответ только ваш настоящий друг — например, вспоминаете события вечеринки два года назад. Если собеседник не может ответить, становится ясно, что это мошенник. В данном случае вопрос про вечеринку является способом проверки личности, то есть аутентификацией.

В веб-приложениях аутентификация чаще всего реализуется через проверку логина и пароля. Когда пользователь вводит свои учетные данные, сервер проверяет их соответствие сохраненным в базе данных. Если введенные данные верны, пользователь получает доступ к системе. В противном случае доступ отклоняется.

Вот корректная формулировка без упоминания **JWT** и **OAuth**:

FastAPI поддерживает стандартные схемы аутентификации (через **OpenAPI/Starlette**):

- **HTTP Basic** — базовая проверка логина и пароля.
- **HTTP Bearer** — передача токена в заголовке **Authorization: Bearer ...**.
- **API Key** — ключ приложения в query-параметре, заголовке или **cookie**.
- **OpenID Connect (Discovery)** — подключение внешнего провайдера идентификации по **/.well-known/openid-configuration**.
- **Сессионные cookie** — сессии на базе Starlette **SessionMiddleware** (удобно для web-логина).

Одним из самых простых подходов является базовая аутентификация, с которой мы и будем работать.

Дополнительные [материалы](#) по теме!

Как работает базовая аутентификация?

Базовая аутентификация — это один из самых простых методов проверки подлинности, который используется для защиты ресурсов в веб-приложениях. В этом методе клиент (например, веб-браузер или **API-клиент**) передает свои учетные данные (имя пользователя и пароль) в заголовке HTTP-запроса, в поле **Authorization**. Формат данных в этом заголовке следующий:

Authorization: Basic <credentials>, где **<credentials>** — это закодированная строка, состоящая из имени пользователя и пароля, разделенных двоеточием. Эта строка кодируется в формате **Base64**.

Пример заголовка запроса с базовой аутентификацией:

```
Authorization: Basic dXNlcm5hbWU6cGFzc3dvcmQ=
```

Когда сервер получает запрос, он извлекает и декодирует строку из заголовка, разделяет имя пользователя и пароль, а затем проверяет их на соответствие сохраненным данным. Если введенные данные совпадают с теми, что есть в базе данных или у поставщика аутентификации, сервер предоставляет доступ к защищенному ресурсу. В противном случае, если данные некорректны, сервер отклоняет запрос и отправляет ошибку, обычно с кодом **401 (Unauthorized)**.

FastAPI облегчает работу с базовой аутентификацией, предоставляя встроенные компоненты для обработки этого типа аутентификации. Это позволяет значительно упростить процесс реализации и интеграции базовой аутентификации в ваше приложение.

Реализация базовой аутентификации в FastAPI

Чтобы реализовать базовую аутентификацию в **FastAPI**, нужно выполнить несколько простых шагов:

Шаг 1: Импорт зависимостей

```
from fastapi import FastAPI, Depends, status, HTTPException
from pydantic import BaseModel
from fastapi.security import HTTPBasic, HTTPBasicCredentials
```

Импортируем необходимые компоненты, включая **FastAPI**, **Depends**, **status**, **HTTPException** и классы для работы с базовой аутентификацией — **HTTPBasic** и **HTTPBasicCredentials**.

Шаг 2: Создайте приложение FastAPI и экземпляр HTTPBasic

```
app = FastAPI()
security = HTTPBasic()
```

Создаем приложение **FastAPI** и экземпляр **HTTPBasic**, который будет использоваться для извлечения учетных данных из запросов.

Шаг 3: Создайте модель пользователя

```
class User(BaseModel):
    username: str
    password: str

# Симуляция базы данных в виде списка объектов пользователей
USER_DATA = [
    User(**{"username": "user1", "password": "pass1"}),
    User(**{"username": "user2", "password": "pass2"})
]
```

Здесь создается модель **User**, описывающая структуру данных пользователя, и простая симуляция базы данных в виде списка объектов.

Шаг 4: Определите функцию аутентификации

```
def authenticate_user(credentials: HTTPBasicCredentials =
Depends(security)):
    user = get_user_from_db(credentials.username)
    if user is None or user.password != credentials.password:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid credentials")
    return user
```

Функция **authenticate_user** получает учетные данные из запроса через механизм зависимостей **FastAPI**. Если пользователь не найден или введенный пароль не совпадает с сохраненным, выбрасывается исключение с кодом **401 (Unauthorized)**.

Шаг 5: Задайте логику получения информации о пользователе

```
def get_user_from_db(username: str):
    for user in USER_DATA:
        if user.username == username:
            return user
    return None
```

Функция **get_user_from_db** ищет пользователя по имени в нашей симулированной базе данных. Если пользователь найден, функция возвращает его объект; если нет — возвращает **None**.

Шаг 6: Защитите конечные точки с помощью аутентификации

```
@app.get("/protected_resource/")
def get_protected_resource(user: User = Depends(authenticate_user)):
    return {"message": "You have access to the protected resource!",
            "user_info": user}
```

В данном шаге мы защищаем конечную точку `/protected_resource/`. При каждом запросе к этому ресурсу **FastAPI** сначала выполнит функцию `authenticate_user` для проверки учетных данных, используя механизм зависимости через `Depends(authenticate_user)`. Если аутентификация пройдена успешно, пользователь получает доступ к защищенному ресурсу.

Разбор работы `Depends`

Класс `Depends` говорит обработчику маршрута, что перед выполнением основного кода маршрута нужно сначала обработать зависимость. В него передается объект, который можно вызвать (обычно это функция, но может быть и класс с методом `__call__`). Например:

```
def my_dependency():
    return "Зависимость выполнена"

@app.get("/")
def read_root(data=Depends(my_dependency)):
    return {"message": data}
```

В этом примере `my_dependency` вызывается автоматически перед выполнением `read_root`, а её результат передается в параметр `data`.

В нашем коде мы используем `Depends(authenticate_user)`. **FastAPI** перед выполнением маршрута `/protected_resource/` вызывает функцию `authenticate_user`, но эта функция сама имеет зависимость — `security`, являющийся экземпляром `HTTPBasic()`. Этот класс проверяет заголовок `Authorization`, сверяет схему аутентификации (`Basic`) и учетные данные. Если они отсутствуют или неверны — выдается ошибка. Если всё корректно, то логин и пароль передаются в `authenticate_user`, который уже проверяет пользователя в нашей базе.

Соображения безопасности

Базовая аутентификация — это простой и удобный способ реализации аутентификации, но у нее есть серьезные ограничения с точки зрения безопасности. Основная проблема заключается в том, что учетные данные (имя пользователя и пароль) передаются в запросах в виде обычного текста, что делает их уязвимыми для перехвата. Особенно это опасно при использовании незащищенных соединений (например, без `HTTPS`).

Чтобы повысить безопасность, следует рассмотреть более защищенные способы аутентификации. Одним из таких является аутентификация на основе `JWT (JSON Web Token)`, которая позволяет безопасно передавать информацию о пользователе в виде токенов, которые могут быть проверены на сервере. Еще один популярный вариант — интеграция с внешними поставщиками

аутентификации, такими как **OAuth**, который обеспечивает более сложные механизмы безопасности, например, аутентификацию через **Google** или **Facebook**.

На данном занятии мы рассмотрели, как реализовать базовую аутентификацию в **FastAPI**. Мы изучили, как работает базовая аутентификация, и пошагово внедрили ее в приложение для защиты определенной конечной точки. Однако, учитывая ограничения этого метода, не забывайте тщательно подходить к вопросам безопасности в реальных приложениях.

Что такое аутентификация на основе JWT?

JWT расшифровывается как веб-токен **JSON (JSON Web Token)** и является популярным методом реализации аутентификации в веб-приложениях. В отличие от базовой аутентификации, где серверу необходимо поддерживать состояние (обычно с помощью сессионных файлов **cookie** или токенов), **JWT** позволяет осуществлять аутентификацию без сохранения состояния.

Когда мы говорим о "без сохранения состояния", это значит, что серверу не нужно хранить информацию о пользователях между запросами. Вместо этого вся информация (например, идентификатор пользователя) хранится непосредственно в токене. Это делает систему более масштабируемой и гибкой, особенно для распределенных приложений или микросервисов.

Также стоит отметить, что **JWT** может использоваться для аутентификации в многосерверных приложениях, поскольку сервер не зависит от централизованного хранилища сессий. В каждом запросе клиент передает токен, который сервер проверяет, не обращаясь к базе данных для поиска пользователя.

Таким образом, аутентификация на основе **JWT** — это безопасный и эффективный способ обеспечения доступа к защищенным ресурсам в веб-приложениях, и с каждым годом этот подход становится все более популярным.

Как работает JWT?

JWT (JSON Web Token) — это объект **JSON** с цифровой подписью, который содержит утверждения о пользователе и дополнительную информацию. Утверждения (**claims**) — это данные о сущности, обычно о пользователе, и других данных, которые могут быть полезны.

JWT состоит из трех частей, разделённых точками: заголовка (**header**), полезной нагрузки (**payload**) и подписи (**signature**):

1. **Заголовок (Header):** Содержит метаданные о токене, такие как алгоритм подписи (например, **HMAC SHA256** или **RSA**).
2. **Полезная нагрузка (Payload):** Содержит утверждения, которые могут включать информацию о пользователе, такие как имя или роль, а также дополнительную информацию. Эти данные не зашифрованы, а только кодируются, поэтому не следует хранить конфиденциальную информацию в этой части.
3. **Подпись (Signature):** Генерируется с использованием секретного ключа и алгоритма подписи. Это гарантирует, что токен не был изменён, и что он был создан авторизованным сервером.

Прочитать подробнее про **JWT** можно на [Habr](#)!

Посмотрим на пример создания и декодирования **JWT-токена**:

```
import jwt # тут используем библиотеку PyJWT

# Секретный ключ для подписи и верификации токенов JWT
SECRET_KEY = "mysecretkey" # в реальной практике используем что-нибудь
вроде команды Bash (Linux) 'openssl rand -hex 32' и храним очень защищённо
ALGORITHM = "HS256" # плюс в реальной жизни устанавливаем "время жизни"
токена

# Пример информации из БД
USERS_DATA = [
    {"username": "admin", "password": "adminpass"}
] # в реальной БД храним только ХЭШИ паролей (например, с помощью
библиотеки 'passlib') + соль (известная только нам добавка к паролю)

# Функция для создания JWT токена
def create_jwt_token(data: dict):
    return jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM) # кодируем
токен, передавая в него наш словарь с нужной информацией

# Функция получения User'a по токenu
def get_user_from_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM]) #
декодируем токен
        return payload.get("sub") # извлекаем утверждение о пользователе
(subject); можем также использовать другие данные, например, "iss"
(issuer) или "exp" (expiration time)
    except jwt.ExpiredSignatureError:
        pass # логика обработки ошибки истечения срока действия токена
    except jwt.InvalidTokenError:
        pass # логика обработки ошибки декодирования токена

# Функция для получения пользовательских данных на основе имени
пользователя
def get_user(username: str):
    for user in USERS_DATA:
        if user.get("username") == username:
            return user
    return None

# Пример использования
# Закодируем токен с утверждением о пользователе
token = create_jwt_token({"sub": "admin"})

print(token) # Выведем токен JWT для просмотра

# Декодируем токен и извлекаем информацию о пользователе
username = get_user_from_token(token)

print(username) # Проверим, что мы извлекли правильное имя пользователя

# Ищем пользователя по имени в нашей базе данных
```

```
current_user = get_user(username)

print(current_user) # Проверяем, что нашли нужного пользователя
```

Пояснение:

- **Заголовок в JWT** обычно содержит информацию об алгоритме подписи, например, **HS256**.
- **Полезная нагрузка** содержит информацию о пользователе — в нашем примере это **"sub": "admin"**.
- **Подпись** генерируется с помощью секретного ключа для защиты от подделки токенов.

В реальной жизни для хранения паролей следует использовать их хэшированные версии, а не хранить в базе данных в открытом виде.

Рабочий процесс JWT

Рабочий процесс аутентификации на основе **JWT** выглядит следующим образом:

1. **Аутентификация пользователя.** Когда пользователь предоставляет действительные учетные данные (например, имя пользователя и пароль) для входа в систему, сервер проверяет их. Если данные верны, сервер генерирует **JWT**.
2. **Отправка JWT клиенту.** Сервер отправляет **JWT** обратно клиенту, обычно в заголовке **Authorization** ответа. Токен передаётся в формате **Bearer <token>**.
3. **Последующие запросы.** Для всех последующих запросов, требующих аутентификации, клиент отправляет **JWT** в заголовке **Authorization**. Сервер проверяет подпись **JWT** с помощью секретного ключа. Если подпись действительна, сервер извлекает информацию о пользователе из утверждений и разрешает доступ к запрашиваемым ресурсам.

Пример с использованием **JWT**:

- Клиент отправляет запрос с логином и паролем.
- Сервер генерирует и отправляет **JWT**, который клиент хранит на своей стороне.
- Для доступа к защищённым ресурсам клиент в каждом запросе отправляет **JWT** в заголовке **Authorization**.
- Сервер проверяет подпись токена, извлекает информацию о пользователе из полезной нагрузки и, если токен действителен, позволяет доступ.

Преимущества аутентификации на основе JWT

- **Без сохранения состояния:** Серверу не нужно хранить информацию о сеансе, что делает его более масштабируемым для распределенных систем. Все необходимые данные находятся в токене, и сервер может работать без необходимости отслеживать состояние каждого пользователя.
- **Междоменность (кросс-доменность):** Поскольку **JWT** обычно отправляются в заголовке **Authorization**, они хорошо работают с **CORS (Cross-Origin Resource Sharing)** и могут использоваться для аутентификации в различных доменах, что делает их удобными для приложений, работающих с несколькими доменами.

- **Пользовательские утверждения:** Вы можете добавлять пользовательские утверждения в полезную нагрузку токена, обеспечивая гибкое хранение информации, такой как роль пользователя, настройки и другие данные, что позволяет более точно контролировать доступ и персонализировать опыт пользователя.

Соображения безопасности

Хотя аутентификация на основе **JWT** дает множество преимуществ, она также сопряжена с важными соображениями безопасности:

- **Срок действия токена:** Установите подходящее время истечения срока действия токена. Это ограничит время, в течение которого злоумышленник может использовать украденный токен. Чем короче срок действия токена, тем меньше времени у хакера на атаки.
- **Используйте HTTPS:** Всегда используйте **HTTPS** для шифрования связи между клиентом и сервером. Это предотвратит перехват токенов и других чувствительных данных во время передачи.
- **Защита секретного ключа:** Секретный ключ, используемый для подписи и верификации токенов, должен быть тщательно защищен. Компрометация ключа может привести к серьезным уязвимостям в системе безопасности.
- **Размер токена:** Помните, что **JWT** отправляется с каждым запросом, что может увеличить нагрузку на сеть. Избегайте включения лишних данных в полезную нагрузку, особенно если они конфиденциальны, чтобы не перегружать запросы и минимизировать риски утечек.

На занятии мы рассмотрим управление доступом на основе ролей, которое позволяет детально управлять разрешениями в вашем приложении **FastAPI**.

Примечание: Имейте в виду, что для применения **JWT** в производственной среде могут потребоваться дополнительные механизмы, такие как отзыв токенов и их обновление, чтобы обеспечить более высокий уровень безопасности.

Пример более продвинутой защиты

Иногда требуется быстро настроить аутентификацию с использованием имени пользователя и пароля, например, при разработке внутри компании. Для этого можно использовать возможности **FastAPI** для работы с **OAuth2**. В этом примере мы демонстрируем, как с помощью **OAuth2PasswordBearer** реализовать получение **JWT-токена** и защиту маршрутов, а также даём несколько пояснений по безопасности и обработке ошибок. Мы разделим проект на несколько файлов, чтобы было легче работать с ним.

Структура проекта:

```

├── main.py           # Основной файл с FastAPI приложением
├── security.py       # Функции для работы с JWT и аутентификацией
├── models.py         # Pydantic схемы для данных
└── db.py             # Файл с "базой данных" для хранения пользователей

```

Код для каждого файла:

security.py:

```
# security.py
import jwt
import datetime
from fastapi import Depends
from fastapi.security import OAuth2PasswordBearer
from typing import Dict

# OAuth2PasswordBearer извлекает токен из заголовка "Authorization: Bearer
<token>"
# Параметр tokenUrl указывает маршрут, по которому клиенты смогут получить
токен
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

SECRET_KEY = "mysecretkey" # В реальной практике генерируйте ключ,
например, с помощью 'openssl rand -hex 32', и храните его в безопасности
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 15 # Время жизни токена

# Функция для создания JWT токена с заданным временем жизни
def create_jwt_token(data: Dict):
    """
    Функция для создания JWT токена. Мы копируем входные данные, добавляем
    время истечения и кодируем токен.
    """
    to_encode = data.copy() # Копируем данные, чтобы не изменить исходный
    словарь
    expire = datetime.datetime.utcnow() +
datetime.timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES) # Задаем время
истечения токена
    to_encode.update({"exp": expire}) # Добавляем время истечения в
данные токена
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM) #
Кодируем токен с использованием секретного ключа и алгоритма

# Функция для получения пользователя из токена
def get_user_from_token(token: str = Depends(oauth2_scheme)):
    """
    Функция для извлечения информации о пользователе из токена. Проверяем
    токен и извлекаем утверждение о пользователе.
    """
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM]) #
Декодируем токен с помощью секретного ключа
        return payload.get("sub") # Возвращаем утверждение о пользователе
(subject) из полезной нагрузки
    except jwt.ExpiredSignatureError:
        pass # Обработка ошибки истечения срока действия токена
    except jwt.InvalidTokenError:
        pass # Обработка ошибки недействительного токена
```

models.py

```
# models.py
from pydantic import BaseModel

class User(BaseModel):
    username: str
    password: str
```

db.py

```
# db.py
# Симуляция базы данных пользователей

USERS_DATA = [
    {"username": "admin", "password": "adminpass"} # В реальной базе
данных пароли должны храниться в виде хэшей
]

def get_user(username: str):
    """
    Функция для поиска пользователя по имени пользователя.
    В реальном проекте это должно быть запросом к базе данных.
    """
    for user in USERS_DATA:
        if user.get("username") == username:
            return user
    return None
```

main.py

```
# main.py
from fastapi import FastAPI, Depends
from security import create_jwt_token, get_user_from_token
from models import User
from db import get_user, USERS_DATA

app = FastAPI()

# Маршрут для аутентификации пользователя.
# Если данные правильные, возвращается JWT токен.
@app.post("/login")
async def login(user_in: User):
    """
    Этот маршрут проверяет учетные данные пользователя и возвращает JWT
    токен, если данные правильные.
    """
    # В реальной практике пароли необходимо сравнивать с хэшами, а не в
```

```
открытом виде
    for user in USERS_DATA:
        if user.get("username") == user_in.username and
user.get("password") == user_in.password:
            # Если проверка прошла успешно, генерируем токен для
пользователя
            token = create_jwt_token({"sub": user_in.username}) # "sub" –
это subject, в нашем случае имя пользователя
            return {"access_token": token, "token_type": "bearer"}
        # Если данные неверные, возвращаем ошибку
        return {"error": "Invalid credentials"}

# Защищённый маршрут, который возвращает информацию о пользователе,
# если токен в запросе действителен.
@app.get("/about_me")
async def about_me(current_user: str = Depends(get_user_from_token)):
    """
    Этот маршрут защищен и требует токен. Если токен действителен, мы
возвращаем информацию о пользователе.
    """
    user = get_user(current_user)
    if user:
        return user
    # Если пользователь не найден, возвращаем ошибку
    return {"error": "User not found"}
```

Объяснение:

- **/login:**
 - Принимаем данные пользователя, проверяем их с теми, что есть в базе данных. В реальном проекте пароли должны сравниваться через хэш.
 - Если данные правильные, генерируем **JWT** токен с помощью функции `create_jwt_token`, в которой добавляется время истечения токена.
- **/about_me:**
 - Защищённый маршрут, который проверяет токен через функцию `get_user_from_token`. Если токен действителен, извлекается имя пользователя, и данные о нем возвращаются.
- **create_jwt_token:**
 - Мы копируем данные (`data.copy()`), чтобы не изменить оригинальный словарь. После этого добавляем информацию о времени истечения токена, а затем кодируем всё это в **JWT** токен с использованием секретного ключа.
- **get_user_from_token:**
 - Эта функция извлекает токен из запроса, декодирует его, чтобы извлечь информацию о пользователе (в нашем случае это имя пользователя, которое хранится в поле `sub`).

Дополнительные рекомендации:

- В реальной практике пароли должны храниться в базе данных в виде хэшей. Для этого удобно использовать библиотеку `passlib`.
- Мы также рекомендуем использовать **HTTPS** для защиты передаваемых данных.
- Обработка ошибок токенов в реальных проектах должна быть более детализированной, например, можно предоставить пользователю возможность обновить токен через специальный маршрут.

Дополнительно рекомендуем ознакомиться с официальной документацией по безопасности FastAPI:

- [Безопасность в FastAPI](#);
- [Продвинутая безопасность в FastAPI](#).

Made with ❤️ by **dvOretsky**